SEMTM0051 — Foundations of Artificial Intelligence

Backpropagation

Instructor: Roberto Colomboni version of 21 October 2026

In Lecture 4, we followed the calculations that turn an input into a network output. We also chose a loss that compares that output with the observed target. To run gradient descent, we now need to find how the loss changes when we change each weight or bias.

We will begin with a small network for which the chain rule from Lecture 1 is enough to calculate every parameter derivative. Calculating these derivatives separately would repeat many of the same steps. *Backpropagation* organises the calculation so that we can compute an intermediate derivative once and use it wherever it is needed. We will then use the scalar example to read the corresponding formulas and diagrams for wider networks, including the network from Lecture 4. The main text explains how to use these formulas. An optional appendix gives the multivariable chain rule and the mathematical derivations. We first calculate the loss derivatives for one observation. Section 4 then explains how to combine these calculations to obtain the gradient for a whole dataset.

We keep the notation of that lecture. Layer r has weights $\boldsymbol{W}_r$ and biases $\boldsymbol{b}_r$, its preactivations are $\boldsymbol{a}_r(\boldsymbol{x})$, and its hidden features are $\boldsymbol{\phi}_r(\boldsymbol{x})$. Inputs and activations are rows, and a weight matrix has one row per input coordinate and one column per output coordinate. The intermediate functions depend on the current parameters, as before, although, to declutter the notation, their subscripts do not include $\boldsymbol{\theta}$.

1 A scalar network that we can calculate by hand

Our aim in this section is to calculate all the partial derivatives needed for one gradient descent step and see how their calculation can be organised to avoid repeating work. We begin with a network for which the chain rule for functions of one variable from Lecture 1 is sufficient.

1.1 The forward pass

For concreteness, we begin with an example of a network containing three layers: two hidden layers and one output layer. The input has one coordinate, and each layer has one unit. The weights, biases, preactivations and activations are therefore all scalars. We write the weights and biases as w_r, b_r, and the hidden feature values as $\phi_r(x)$. We also define

$$\phi_0(x) \coloneqq x.$$

This is the input itself. With this convention, $\phi_{r-1}(x)$ denotes the input to layer r, for $r = 1, 2, 3$. The calculation is

$$\begin{aligned} a_1(x) &= \phi_0(x)\, w_1 + b_1, & \phi_1(x) &= \mathrm{ReLU}(a_1(x)), \\ a_2(x) &= \phi_1(x)\, w_2 + b_2, & \phi_2(x) &= \mathrm{ReLU}(a_2(x)), \\ a_3(x) &= \phi_2(x)\, w_3 + b_3, & f_{\boldsymbol{\theta}}(x) &= a_3(x). \end{aligned} \tag{1}$$

Since the output layer does not apply ReLU, its affine output $a_3(x)$ is also the network prediction $f_{\boldsymbol{\theta}}(x)$.

For this example, we collect the parameters in the order $\boldsymbol{\theta} = (b_1, w_1, b_2, w_2, b_3, w_3)^\mathsf{T}$. We use a dataset containing just one fixed observation (x, y). The squared-loss objective is consequently

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = \ell_{\text{sq}}\big(f_{\boldsymbol{\theta}}(x), y\big) = \big(a_3(x) - y\big)^2. \tag{2}$$

The objective is a function of the parameters. The input and target remain fixed while we calculate its gradient.

Take, for example,

$$x = 2, \quad y = -2, \qquad w_1 = 1,\ b_1 = 1, \quad w_2 = 2,\ b_2 = -1, \quad w_3 = -1,\ b_3 = 2. \tag{3}$$

Here $\phi_0(x) = x = 2$. The first affine calculation gives $a_1(x) = \phi_0(x)\, w_1 + b_1 = 2(1) + 1 = 3$, so $\phi_1(x) = 3$. The next gives $a_2(x) = 3(2) - 1 = 5$, so $\phi_2(x) = 5$. Finally,

$$a_3(x) = 5(-1) + 2 = -3, \qquad \mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = (-3 + 2)^2 = 1.$$

Figure 1 follows this calculation from left to right. The boxes in the main row show the input, the intermediate values, the prediction and the loss, with the numerical value written below each quantity. The arrows indicate which earlier values are used in each calculation. The weights and biases enter at the affine operations, while the target enters at the loss.

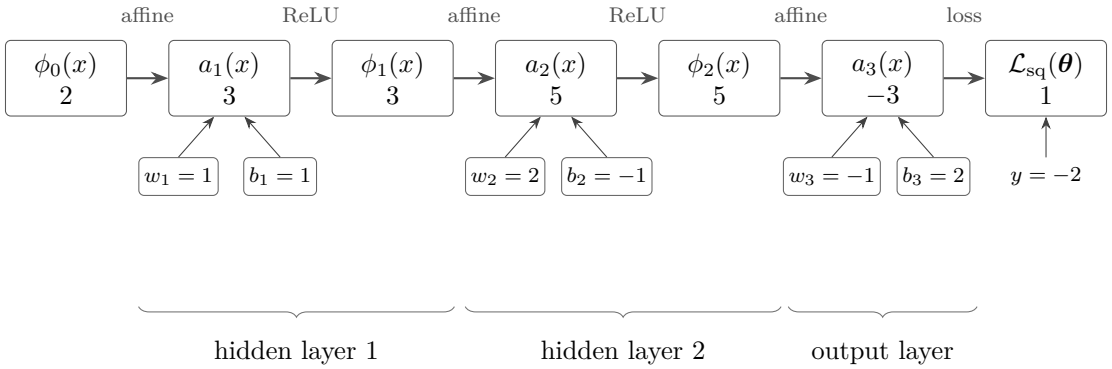


Figure 1: The scalar forward pass for the observation and parameters in (3). The lower braces identify the two hidden layers and the output layer. The loss is calculated after the network has produced its prediction.

1.2 Identifying the remaining calculations

We now organise the forward calculation to prepare for backpropagation. Starting from each preactivation, we identify the function that performs all the remaining operations needed to obtain the loss. For layer r, we call this function J_r.

Throughout these notes and the slides, we write J_r to keep the notation light, leaving its dependence on the target and the parameters of layers after r implicit. When differentiating this function, we vary its argument while holding those quantities fixed.

Why introduce these functions? As we will show in the next subsection, differentiating J_r with respect to its argument and evaluating that derivative at $a_r(x)$ gives the loss derivative with respect to b_r. Multiplying the same quantity by the layer input $\phi_{r-1}(x)$ gives the loss derivative with respect to w_r. We can therefore calculate this quantity once and reuse it for both parameters. We will also use it to calculate the corresponding derivative for the preceding layer.

In each definition below, u is the argument of the remaining function: it represents a possible preactivation value. The target and the parameters used by the remaining operations are held fixed. After defining the function, we evaluate it at the particular value $u = a_r(x)$ produced by the forward pass.

Start at the output $a_3(x)$. Given a possible output value u, the remaining calculation subtracts y and squares the difference. We therefore define

$$J_3 : \mathbb{R} \longrightarrow \mathbb{R}, \qquad J_3(u) = (u - y)^2.$$

The index 3 identifies the output layer because this network has three layers. This particular function uses only y and ignores the weights and biases in $\boldsymbol{\theta}$.

Evaluating this function at the actual output gives

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = \big(a_3(x) - y\big)^2 = J_3\big(a_3(x)\big).$$

Now start at $a_2(x)$. We still need to apply ReLU, perform the final affine operation, and evaluate the loss. For a possible preactivation value u, these operations define

$$J_2 : \mathbb{R} \longrightarrow \mathbb{R}, \qquad J_2(u) = \big(\text{ReLU}(u)\, w_3 + b_3 - y\big)^2 = J_3\big(\text{ReLU}(u)\, w_3 + b_3\big). \tag{4}$$

Thus J_2 depends only on w_3, b_3, y. It does not use x or the parameters w_1, b_1, w_2, b_2, which are used to calculate $a_2(x)$.

Recalling that $a_3(x) = \text{ReLU}\big(a_2(x)\big)\, w_3 + b_3$, we obtain

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = J_3\big(a_3(x)\big) = J_3\Big(\text{ReLU}\big(a_2(x)\big)\, w_3 + b_3\Big) = J_2\big(a_2(x)\big).$$

Keeping w_3, b_3, y fixed, changing an earlier parameter can change $a_2(x)$, and therefore the value $J_2(a_2(x))$, without changing the function J_2 itself.

Finally, start at $a_1(x)$. We first apply ReLU and the second affine operation. The resulting value can then be passed to J_2, which completes the calculation. We therefore define

$$J_1 : \mathbb{R} \longrightarrow \mathbb{R}, \qquad J_1(u) = J_2\big(\text{ReLU}(u)\, w_2 + b_2\big). \tag{5}$$

This function uses w_2, b_2, w_3, b_3, y. It does not use x, w_1, b_1: their effect enters through the supplied preactivation value $a_1(x)$.

Recalling that $a_2(x) = \text{ReLU}\big(a_1(x)\big)\, w_2 + b_2$, we obtain

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = J_2\big(a_2(x)\big) = J_2\Big(\text{ReLU}\big(a_1(x)\big)\, w_2 + b_2\Big) = J_1\big(a_1(x)\big).$$

At their respective forward values, all three remaining functions therefore give the same loss:

$$J_r\big(a_r(x)\big) = \mathcal{L}_{\text{sq}}(\boldsymbol{\theta}), \qquad r = 1, 2, 3. \tag{6}$$

Here $a_r(x)$ is a number produced by the forward pass, J_r is a function, and $J_r(a_r(x))$ is the value obtained by evaluating that function at that number. For example, our forward pass gave $a_2(x) = 5$, and completing the calculation from this value gives $J_2(5) = 1$.

The definitions also show how the remaining calculations fit together. To evaluate J_1, we perform the operations that produce the next preactivation and then use J_2. In turn, J_2 performs the next operations and then uses J_3. These compositions will let us reuse derivatives when we work backwards.

Figure 2 shows the same calculation as Figure 1, now with coloured regions identifying the operations included in each remaining function. The box marked "input" in the matching colour shows the forward value supplied as that function's argument.

Each region uses only the supplied input value and the parameters and target shown inside it. It does not need to know how that input was produced: all dependence on the earlier calculations enters through that value. Once the input has been supplied, we can evaluate the remaining function without repeating or referring to those earlier calculations. The nested regions show how each remaining function uses the one that follows it.

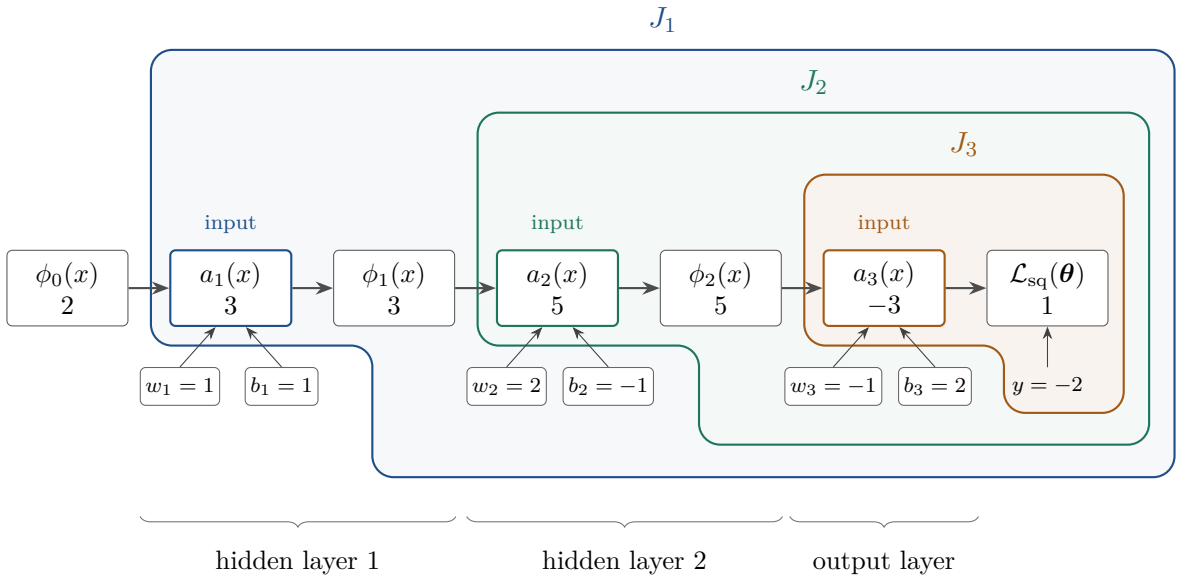


Figure 2: The forward pass at the parameters in (3). Each coloured region represents one remaining function, evaluated at the input marked in the same colour. Its calculation uses that input and the quantities inside the region. The earlier quantities affect the result only through the supplied input value.

1.3 Finding and reusing the shared derivatives

We now use these functions to calculate the six parameter derivatives, beginning with the output layer. At each layer, we will identify the derivative shared by its weight and bias calculations and use it to continue backwards. All derivatives are evaluated at the parameter values used in the forward pass. We update the parameters only after calculating the gradient. Assume that $a_1(x)$ and $a_2(x)$ are nonzero, as they are in our numerical example. This ensures that the ReLU derivatives used below exist.

Recall from Lecture 4 that $\text{ReLU}'(u) = \text{step}(u)$ for $u \neq 0$, where the Heaviside step function is

$$\text{step}(u) = \begin{cases} 0, & u \leq 0, \\ 1, & u > 0. \end{cases}$$

We will use this function to write the backward calculations. It keeps a value when the preactivation is positive and sets it to zero otherwise. Its use at a zero preactivation is a computational convention, discussed in Section 1.5.

First consider w_3. We have already written the loss as

$$\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = J_3(a_3(x)).$$

The function $J_3(u) = (u - y)^2$ depends only on the fixed target y. Changing w_3 therefore changes its argument $a_3(x)$, while the function itself remains fixed. Its ordinary derivative is

$$J_3'(u) = \frac{\mathrm{d}}{\mathrm{d}u}(u - y)^2 = 2\,(u - y).$$

Evaluating this derivative at $u = a_3(x)$ gives the starting value for the backward calculation.

In $a_3(x) = \phi_2(x)\,w_3 + b_3$, the coefficient $\phi_2(x)$ does not depend on w_3. Applying the chain rule to the composition above therefore gives

$$\begin{aligned}\frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial w_3}(\boldsymbol{\theta}) &= J_3'(a_3(x))\,\frac{\partial a_3(x)}{\partial w_3} \\ &= J_3'(a_3(x))\,\frac{\partial}{\partial w_3}(\phi_2(x)\,w_3 + b_3) \\ &= J_3'(a_3(x))\,\phi_2(x).\end{aligned} \tag{7}$$

Thus $J_3'(a_3(x))$ appears because we differentiate the outer function in the composition.

For b_3, the derivative of $a_3(x)$ is one. Applying the same chain rule gives

$$\frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial b_3}(\boldsymbol{\theta}) = J_3'(a_3(x))\,\frac{\partial a_3(x)}{\partial b_3} = J_3'(a_3(x)).$$

Both parameter derivatives therefore use the same evaluated derivative of J_3. We can calculate this value once, use it directly for b_3, and multiply it by $\phi_2(x)$ for w_3.

Next consider w_2 and b_2. These parameters determine $a_2(x) = \phi_1(x)\,w_2 + b_2$, and J_2 completes the calculation from $a_2(x)$. We therefore need the derivative of J_2 at this forward value. From (4), for $u \neq 0$, the ordinary chain rule gives

$$\begin{aligned}J_2'(u) &= J_3'(\mathrm{ReLU}(u)\,w_3 + b_3)\,\frac{\mathrm{d}}{\mathrm{d}u}\,(\mathrm{ReLU}(u)\,w_3 + b_3) \\ &= J_3'(\mathrm{ReLU}(u)\,w_3 + b_3)\,w_3\;\mathrm{ReLU}'(u).\end{aligned}$$

At $u = a_2(x)$, the argument of J_3' becomes $a_3(x)$. Consequently,

$$J_2'(a_2(x)) = \underbrace{J_3'(a_3(x))}_{\text{already calculated}}\;\underbrace{w_3}_{\text{affine derivative}}\;\underbrace{\mathrm{step}(a_2(x))}_{\text{ReLU derivative}}. \tag{8}$$

Equation (8) tells us how to calculate $J_2'(a_2(x))$ using $J_3'(a_3(x))$, which we have already calculated. We multiply the latter by w_3, the derivative of the final affine function with respect to its input, and by $\mathrm{step}(a_2(x))$, the derivative of ReLU at the saved preactivation.

In the forward pass, we used $a_2(x)$ to calculate $a_3(x)$. Here, we use the derivative associated with $a_3(x)$ to calculate the derivative associated with $a_2(x)$. This is the sense in which the calculation proceeds backwards through the layers.

We can now reuse $J_2'(a_2(x))$ to calculate the loss derivatives with respect to w_2 and b_2.

The feature $\phi_1(x)$ and the function J_2 do not depend on w_2 or b_2. The preactivation $a_2(x)$ has derivative $\phi_1(x)$ with respect to w_2 and derivative one with respect to b_2. Applying the chain rule to $\mathcal{L}_{\mathrm{sq}}(\boldsymbol{\theta}) = J_2(a_2(x))$ therefore gives

$$\begin{aligned}\frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial w_2}(\boldsymbol{\theta}) &= J_2'(a_2(x))\,\frac{\partial a_2(x)}{\partial w_2} = J_2'(a_2(x))\,\phi_1(x), \\ \frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial b_2}(\boldsymbol{\theta}) &= J_2'(a_2(x))\,\frac{\partial a_2(x)}{\partial b_2} = J_2'(a_2(x)).\end{aligned} \tag{9}$$

Both derivatives use $J_2'(a_2(x))$, which we have just obtained from the output-layer result. We can therefore reuse that value for both parameters.

Finally, consider w_1 and b_1. The function J_1 completes the calculation from $a_1(x)$. Differentiating (5) near this forward value gives

$$\begin{aligned} J_1'(u) &= J_2'(\mathrm{ReLU}(u)\, w_2 + b_2)\, \frac{\mathrm{d}}{\mathrm{d}u}\, (\mathrm{ReLU}(u)\, w_2 + b_2) \\ &= J_2'(\mathrm{ReLU}(u)\, w_2 + b_2)\, w_2\ \mathrm{ReLU}'(u). \end{aligned}$$

The nonzero forward preactivations ensure differentiability for u sufficiently close to $a_1(x)$. At that forward value, the argument of J_2' is $a_2(x)$, so

$$J_1'(a_1(x)) = \underbrace{J_2'(a_2(x))}_{\text{already calculated}}\ \underbrace{w_2}_{\text{affine derivative}}\ \underbrace{\mathrm{step}(a_1(x))}_{\text{ReLU derivative}}. \tag{10}$$

We reuse the derivative calculated at the second layer and multiply it by $w_2\ \mathrm{step}(a_1(x))$ to obtain the derivative needed for the first layer.

Since $a_1(x) = \phi_0(x)\, w_1 + b_1$, its derivatives with respect to w_1 and b_1 are $\phi_0(x)$ and one. The value $\phi_0(x) = x$ is fixed, and the function J_1 does not depend on these parameters. The last two parameter derivatives are therefore

$$\begin{aligned} \frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial w_1}(\boldsymbol{\theta}) &= J_1'(a_1(x))\, \frac{\partial a_1(x)}{\partial w_1} = J_1'(a_1(x))\, \phi_0(x), \\ \frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial b_1}(\boldsymbol{\theta}) &= J_1'(a_1(x))\, \frac{\partial a_1(x)}{\partial b_1} = J_1'(a_1(x)). \end{aligned} \tag{11}$$

The same procedure applies at every layer. Once we have calculated $J_r'(a_r(x))$, it is already the derivative of the loss with respect to that layer's bias. Multiplying it by the layer's input, $\phi_{r-1}(x)$, gives the derivative with respect to its weight. Thus, for $r = 1, 2, 3$,

$$\begin{aligned} \frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial b_r}(\boldsymbol{\theta}) &= J_r'(a_r(x)), \\ \frac{\partial \mathcal{L}_{\mathrm{sq}}}{\partial w_r}(\boldsymbol{\theta}) &= J_r'(a_r(x))\, \phi_{r-1}(x). \end{aligned} \tag{12}$$

The convention $\phi_0(x) = x$ makes this rule apply to the first layer as well. We first calculate $J_3'(a_3(x))$, reuse it to obtain $J_2'(a_2(x))$, and reuse that result to obtain $J_1'(a_1(x))$. These are intermediate derivatives that we use to obtain the six parameter derivatives needed for gradient descent.

Figure 3 summarises one backward step. The same intermediate derivative gives the two parameter derivatives below it and, when $r > 1$, the derivative for the preceding layer to its left. That new derivative is then reused in the next step.

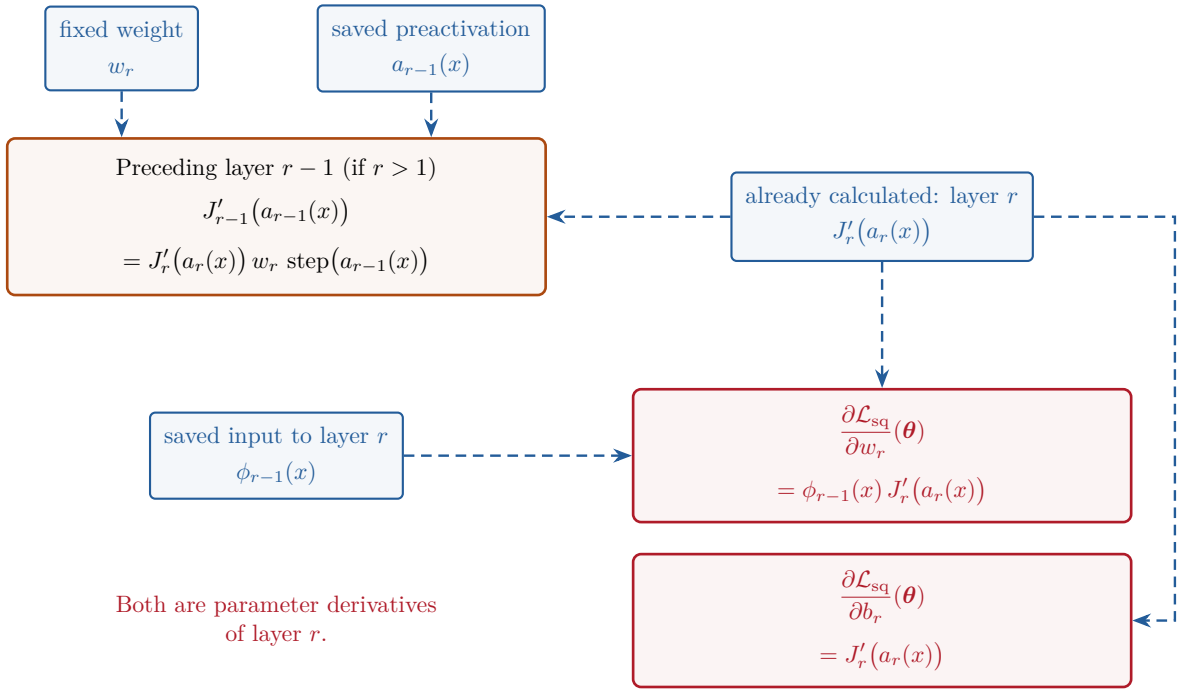


Figure 3: Reusing one intermediate derivative in the scalar network. For $r = 1, 2, 3$, the blue value on the right gives the two red parameter derivatives below it. If $r > 1$, it also gives the orange derivative for the preceding layer, using the fixed weight and saved preactivation above. That orange value becomes the blue input at the next step. Both hidden preactivations are assumed nonzero.

1.4 The backward pass for our numerical example

We now execute this backward calculation at the parameters in (3). The saved preactivations are $a_1(x) = 3$, $a_2(x) = 5$, and $a_3(x) = -3$. The inputs to the three layers are also available: $\phi_0(x) = 2$, $\phi_1(x) = 3$, and $\phi_2(x) = 5$. The target is $y = -2$. Both hidden preactivations are positive, so $\text{step}(3) = \text{step}(5) = 1$. Starting at the output and proceeding towards the input gives

$$J_3'(-3) = 2(-3 + 2) = -2,$$

$$J_2'(5) = \underbrace{(-2)}_{J_3'(-3)}\ \underbrace{(-1)}_{w_3}\ \underbrace{(1)}_{\text{step}(5)} = 2,$$

$$J_1'(3) = \underbrace{2}_{J_2'(5)}\ \underbrace{(2)}_{w_2}\ \underbrace{(1)}_{\text{step}(3)} = 4.$$

For example, at the second layer, $a_2(x) = 5$ is the preactivation, $J_2(5) = 1$ is the resulting loss, and $J_2'(5) = 2$ is the derivative of that remaining loss function. A small increase from the preactivation value 5 therefore increases the loss to first order.

Using these intermediate derivatives and the saved layer inputs in (12) gives the parameter derivatives

$$\begin{aligned}
\frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_3}(\boldsymbol{\theta}) &= (-2)(5) = -10, & \frac{\partial \mathcal{L}_{\text{sq}}}{\partial b_3}(\boldsymbol{\theta}) &= -2,\\
\frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_2}(\boldsymbol{\theta}) &= 2(3) = 6, & \frac{\partial \mathcal{L}_{\text{sq}}}{\partial b_2}(\boldsymbol{\theta}) &= 2,\\
\frac{\partial \mathcal{L}_{\text{sq}}}{\partial w_1}(\boldsymbol{\theta}) &= 4(2) = 8, & \frac{\partial \mathcal{L}_{\text{sq}}}{\partial b_1}(\boldsymbol{\theta}) &= 4.
\end{aligned}$$

Figures 4–6 show the three steps in order. The top row keeps the saved forward values in the same positions throughout. Blue marks values already available that we use in the current step. These may be forward values, a fixed parameter or target, or an intermediate derivative calculated in the preceding step. Orange marks the new intermediate derivative, while red marks the parameter derivatives that we want to obtain. Results from earlier steps remain visible, with lighter outlines.

The new intermediate derivatives are calculated from right to left, but the forward values are all available before this process begins. For example, $\phi_2(x)$ is already known when we calculate the derivative with respect to w_3. The blue dashed arrow shows it entering that calculation alongside $J_3'(a_3(x))$. The weights and biases keep their forward-pass values throughout all three steps.

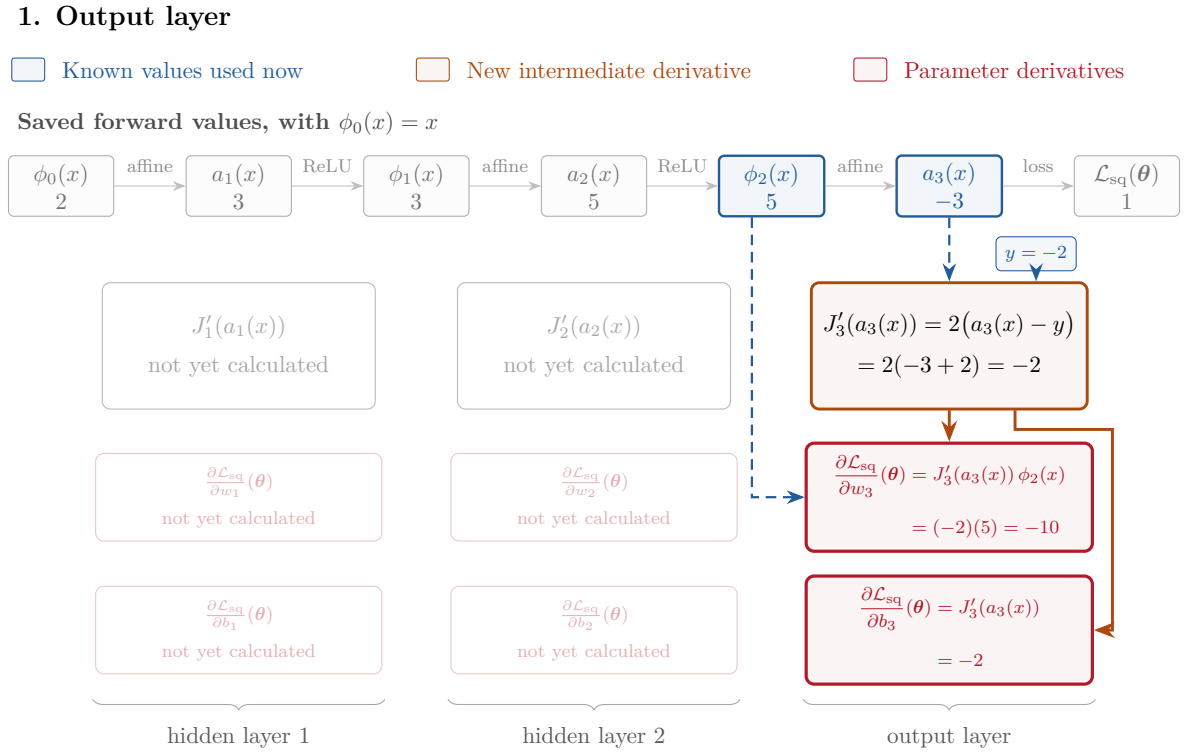


Figure 4: Step 1: the output layer. The prediction $a_3(x)$ and target y give $J_3'(a_3(x))$. This is the derivative with respect to b_3. Multiplying it by the saved input $\phi_2(x)$ gives the derivative with respect to w_3.

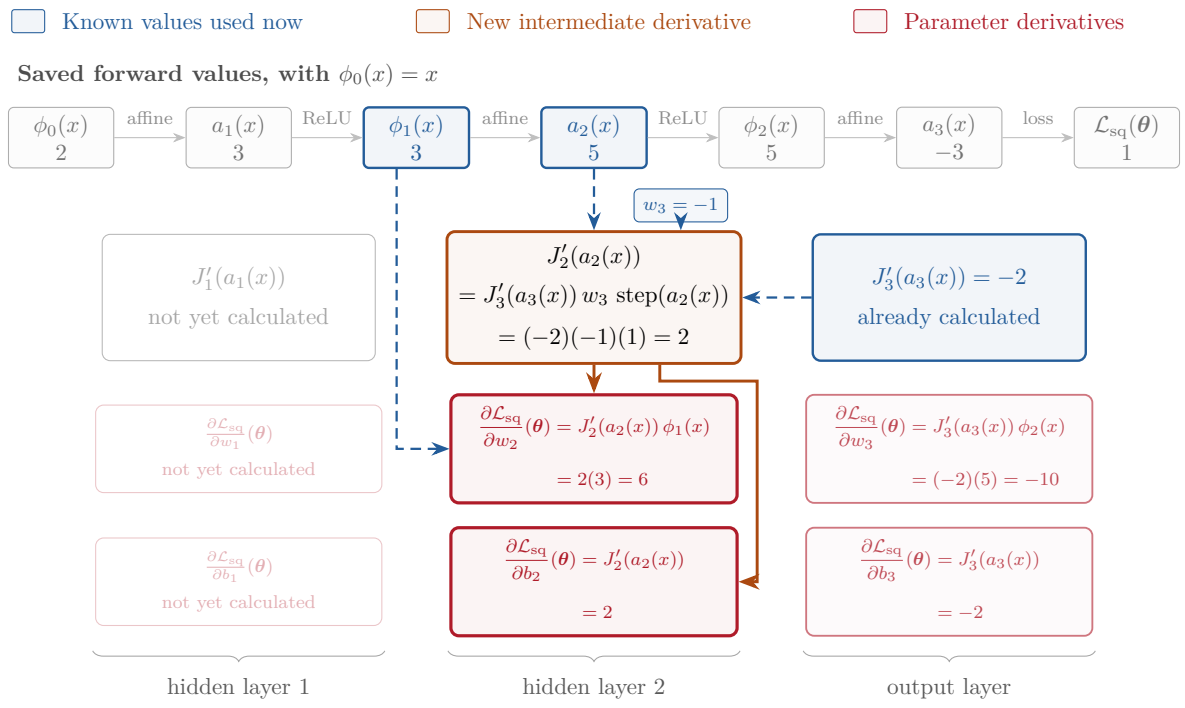


Figure 5: Step 2: the second hidden layer. We reuse $J_3'(a_3(x))$, together with w_3 and $a_2(x)$, to calculate $J_2'(a_2(x))$. This gives the bias derivative directly and, after multiplication by the saved input $\phi_1(x)$, the weight derivative.

3. Hidden layer 1

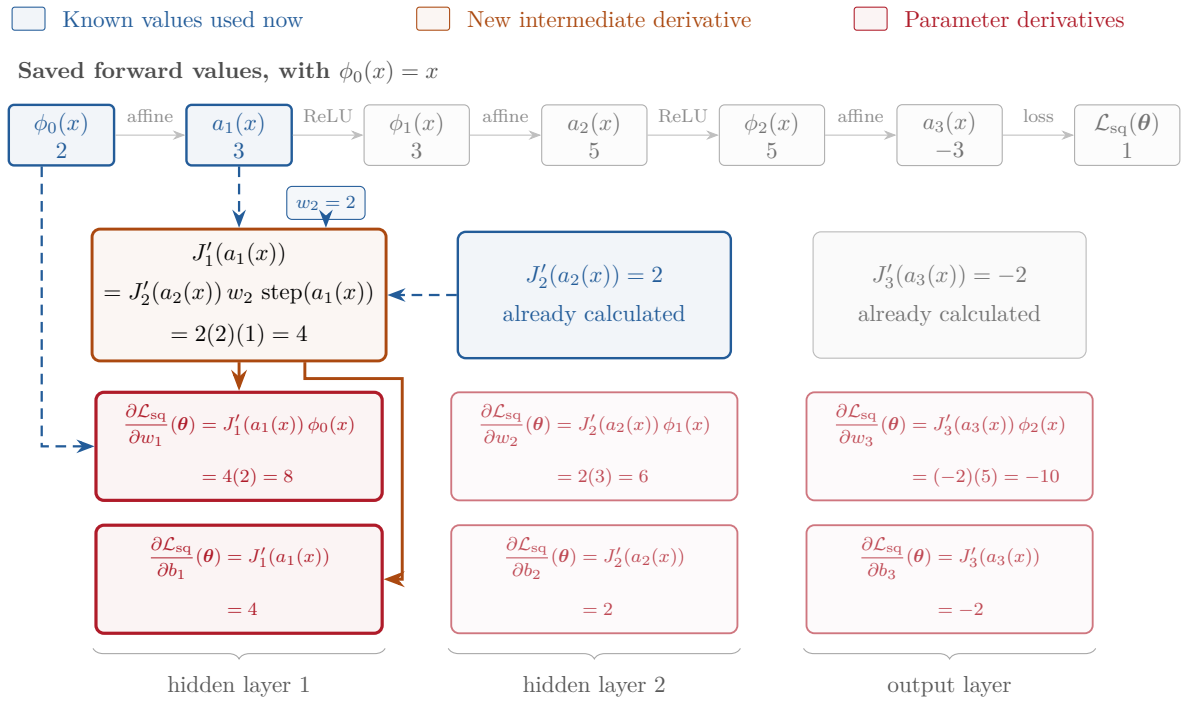


Figure 6: Step 3: the first hidden layer. We reuse $J_2'(a_2(x))$, together with w_2 and $a_1(x)$, to calculate $J_1'(a_1(x))$. The layer input is $\phi_0(x) = x$, so the same rule gives the final two parameter derivatives. All six required derivatives are now shown in red.

The parameter column was ordered as $(b_1, w_1, b_2, w_2, b_3, w_3)^\mathsf{T}$. In that same order, the gradient is

$$\nabla \mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = \begin{pmatrix} 4 & 8 & 2 & 6 & -2 & -10 \end{pmatrix}^\mathsf{T}.$$

We have now obtained the six partial derivatives needed for a gradient descent step. Backpropagation calculated them by reusing the three intermediate derivatives $J_3'(a_3(x))$, $J_2'(a_2(x))$, and $J_1'(a_1(x))$. Gradient descent can then use the resulting gradient to update the weights and biases.

1.5 The convention at a zero preactivation

As discussed in Lecture 4, PyTorch's ReLU implementation uses zero as the multiplier in the backward calculation when the preactivation is zero. We adopt the same convention through $\text{step}(0) = 0$. This multiplier is a computational choice, not an ordinary derivative of ReLU at zero, which does not exist.

2 The same calculation for vectors

The scalar example already contains the organisation of backpropagation: save the forward values, start at the loss, and reuse each remaining-loss derivative as we work backwards. For a wider network, we follow the same steps with rows and matrices. We give the formulas and explain how to use them here; the derivations, including the multivariable chain rule, are optional reading in Appendix A.

2.1 From single values to rows and matrices

As in Lecture 4, L counts all layers, including the output layer, d_r is the width of layer r, and d_0 is the input width. Each layer has its own parameters. The quantities from the scalar example become

Quantity	Scalar network	Wider network
Input to layer r	$\phi_{r-1}(x)$	$\boldsymbol{\phi}_{r-1}(\boldsymbol{x}) \in \mathbb{R}^{1\times d_{r-1}}$
Weights	w_r	$\boldsymbol{W}_r \in \mathbb{R}^{d_{r-1}\times d_r}$
Biases	b_r	$\boldsymbol{b}_r \in \mathbb{R}^{1\times d_r}$
Preactivations	$a_r(x)$	$\boldsymbol{a}_r(\boldsymbol{x}) \in \mathbb{R}^{1\times d_r}$

Instead of one weight, we now have one weight for each connection between an input coordinate and an output coordinate. Each output coordinate has its own bias.

The forward pass keeps the same order of operations. We write $\cdot$ for ordinary matrix multiplication:

$$\begin{aligned} &\boldsymbol{\phi}_0(\boldsymbol{x}) = \boldsymbol{x}, \\ &\boldsymbol{a}_r(\boldsymbol{x}) = \boldsymbol{\phi}_{r-1}(\boldsymbol{x}) \cdot \boldsymbol{W}_r + \boldsymbol{b}_r, && r = 1, \ldots, L, \\ &\boldsymbol{\phi}_r(\boldsymbol{x}) = \mathbf{ReLU}(\boldsymbol{a}_r(\boldsymbol{x})), && r = 1, \ldots, L-1. \end{aligned} \tag{13}$$

Here **ReLU** applies ReLU to each coordinate. The final layer has no ReLU: $\boldsymbol{a}_L(\boldsymbol{x}) = \boldsymbol{f}_{\boldsymbol{\theta}}(\boldsymbol{x})$. For one output, we identify this one-entry row with the scalar prediction $f_{\boldsymbol{\theta}}(\boldsymbol{x})$. The formulas also cover $L = 1$, a single affine output layer with no hidden layers.

2.2 The remaining functions and their gradients

The role of J_r stays the same: it performs all the operations from a possible preactivation to the final loss. Its input is now a row:

$$J_r : \mathbb{R}^{1\times d_r} \longrightarrow \mathbb{R}.$$

For regression with one output, $J_L(\boldsymbol{u}) = (u_1 - y)^2$. For an earlier layer, the definition is the same composition as in the scalar example:

$$J_r(\boldsymbol{u}) = J_{r+1}(\mathbf{ReLU}(\boldsymbol{u}) \cdot \boldsymbol{W}_{r+1} + \boldsymbol{b}_{r+1}), \quad r = L-1, \ldots, 1. \tag{14}$$

We apply ReLU, perform the next affine calculation, and let J_{r+1} complete the remaining operations. The target and the parameters of subsequent layers are fixed in this definition. Section 3 gives the output function for classification.

For one fixed observation, let $\mathcal{L}(\boldsymbol{\theta})$ be the chosen loss objective. Evaluating any remaining function at its actual forward value gives that same loss:

$$J_r(\boldsymbol{a}_r(\boldsymbol{x})) = \mathcal{L}(\boldsymbol{\theta}).$$

As before, $\mathcal{L}$ is a function of the parameters, whereas J_r is a function of a possible preactivation, with the target and the parameters of subsequent layers held fixed.

In the scalar example, we calculated one derivative, $J_r'(a_r(x))$. Now we need one partial derivative for each coordinate of the row argument. Recall from Lecture 1 that the gradient collects these derivatives in the same shape as the input. Thus

$$\nabla J_r(\boldsymbol{a}_r(\boldsymbol{x})) = \left(\left(\frac{\partial J_r}{\partial u_1}\right)(\boldsymbol{a}_r(\boldsymbol{x})) \quad \cdots \quad \left(\frac{\partial J_r}{\partial u_{d_r}}\right)(\boldsymbol{a}_r(\boldsymbol{x}))\right). \tag{15}$$

This is a row of shape $1 \times d_r$. Its entries describe how the remaining loss changes when we vary each input coordinate, holding the other coordinates fixed. This row replaces the single number $J_r'(a_r(x))$ in the backward pass.

As in the scalar example, the derivative formulas assume that every hidden preactivation coordinate is nonzero. At zero, the backward calculation uses the multiplier $\text{step}(0) = 0$, with the interpretation from Section 1.5.

2.3 Distinguishing two kinds of multiplication

The backward pass for rows and matrices uses two different products. We distinguish ordinary matrix multiplication from multiplication of matching entries, and compare their results.

Ordinary matrix multiplication, $\cdot$. For $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ and $\boldsymbol{B} \in \mathbb{R}^{n\times p}$, recall that the ordinary matrix product $\boldsymbol{A} \cdot \boldsymbol{B}$ has shape $m \times p$, with

$$[\boldsymbol{A} \cdot \boldsymbol{B}]_{ij} = \sum_{k=1}^{n} A_{ik} \cdot B_{kj}, \qquad i = 1, \ldots, m, \quad j = 1, \ldots, p.$$

The number of columns of the first matrix must equal the number of rows of the second. We multiply entries from a row of $\boldsymbol{A}$ and a column of $\boldsymbol{B}$, then add the resulting products.

Componentwise multiplication, $\odot$. For two matrices $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{m\times n}$, define their *componentwise product* by

$$[\boldsymbol{A} \odot \boldsymbol{B}]_{ij} \coloneqq A_{ij} \cdot B_{ij}, \qquad i = 1, \dots, m, \quad j = 1, \dots, n. \tag{16}$$

The two matrices must have exactly the same shape. We multiply entries in the same position, without adding across coordinates; the result also has shape $m \times n$. For example, two 2×3 matrices have a componentwise product, but their ordinary matrix product is not defined.

To compare the results when both products are defined, take

$$\boldsymbol{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad \boldsymbol{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

Then

$$\boldsymbol{A} \cdot \boldsymbol{B} = \begin{pmatrix} 19 & 22 \\ 43 & 50 \end{pmatrix}, \qquad \boldsymbol{A} \odot \boldsymbol{B} = \begin{pmatrix} 5 & 12 \\ 21 & 32 \end{pmatrix}.$$

For instance, their top-left entries are calculated differently:

$$[\boldsymbol{A} \cdot \boldsymbol{B}]_{11} = 1 \cdot 5 + 2 \cdot 7 = 19, \qquad [\boldsymbol{A} \odot \boldsymbol{B}]_{11} = 1 \cdot 5 = 5.$$

We also use $\cdot$ for ordinary scalar multiplication and for scalar multiplication of a matrix, which multiplies every entry by that scalar and preserves the matrix's shape.

For two rows of the same shape, the componentwise definition gives

$$\boldsymbol{u} \odot \boldsymbol{v} = \begin{pmatrix} u_1 \cdot v_1 & \cdots & u_m \cdot v_m \end{pmatrix}, \qquad \boldsymbol{u}, \boldsymbol{v} \in \mathbb{R}^{1\times m}.$$

It multiplies matching entries and returns a row of the same shape.

2.4 One backward step

Suppose the gradient $\nabla J_r(\boldsymbol{a}_r(\boldsymbol{x}))$ is already calculated. We use it for the parameters of layer r and, if $r > 1$, to continue to the preceding layer.

The parameter gradients. In the scalar example, the bias derivative was the shared number $J_r'(a_r(x))$, and the weight derivative also included the saved layer input $\phi_{r-1}(x)$. The wider network uses exactly the same ingredients:

$$\boxed{\begin{aligned} \nabla_{\boldsymbol{W}_r} \mathcal{L}(\boldsymbol{\theta}) &= \boldsymbol{\phi}_{r-1}(\boldsymbol{x})^{\mathsf{T}} \cdot \nabla J_r(\boldsymbol{a}_r(\boldsymbol{x})), \\ \nabla_{\boldsymbol{b}_r} \mathcal{L}(\boldsymbol{\theta}) &= \nabla J_r(\boldsymbol{a}_r(\boldsymbol{x})). \end{aligned}} \tag{17}$$

A subscript on ∇ specifies which parameters we differentiate with respect to, holding the others fixed. The derivatives have the same shapes as those parameters: $d_{r-1} \times d_r$ for the weights and $1 \times d_r$ for the biases.

Checking the dimensions. We can check that the weight-gradient formula has the expected dimensions. Its two factors satisfy

$$\boldsymbol{\phi}_{r-1}(\boldsymbol{x})^{\mathsf{T}} \in \mathbb{R}^{d_{r-1}\times 1}, \qquad \nabla J_r(\boldsymbol{a}_r(\boldsymbol{x})) \in \mathbb{R}^{1\times d_r}.$$

The first factor has one column and the second has one row, so their matrix product is defined and satisfies

$$\boldsymbol{\phi}_{r-1}(\boldsymbol{x})^{\mathsf{T}} \cdot \nabla J_r(\boldsymbol{a}_r(\boldsymbol{x})) \in \mathbb{R}^{d_{r-1}\times d_r}.$$

The result has the same shape as $\boldsymbol{W}_r$, as required for a matrix containing one partial derivative for each weight.

Continuing to the preceding layer. In the scalar calculation, we multiplied the shared derivative by w_r, then by $\operatorname{step}(a_{r-1}(x))$. As with **ReLU**, bold type means applying the scalar step function to each coordinate:

$$\mathbf{step}(\boldsymbol{u}) \coloneqq \begin{pmatrix} \operatorname{step}(u_1) & \cdots & \operatorname{step}(u_m) \end{pmatrix}, \qquad \boldsymbol{u} \in \mathbb{R}^{1\times m}. \tag{18}$$

The corresponding formula for rows is

$$\boxed{\nabla J_{r-1}(\boldsymbol{a}_{r-1}(\boldsymbol{x})) = \Big(\nabla J_r(\boldsymbol{a}_r(\boldsymbol{x})) \cdot \boldsymbol{W}_r^{\mathsf{T}}\Big) \odot \mathbf{step}(\boldsymbol{a}_{r-1}(\boldsymbol{x}))}, \qquad r > 1. \tag{19}$$

Checking the dimensions. We can check that both products in the backward formula are well defined. For the matrix product, we have

$$\nabla J_r(\boldsymbol{a}_r(\boldsymbol{x})) \in \mathbb{R}^{1\times d_r}, \qquad \boldsymbol{W}_r^{\mathsf{T}} \in \mathbb{R}^{d_r \times d_{r-1}}.$$

The number of columns of the first factor equals the number of rows of the second, so

$$\nabla J_r(\boldsymbol{a}_r(\boldsymbol{x})) \cdot \boldsymbol{W}_r^{\mathsf{T}} \in \mathbb{R}^{1\times d_{r-1}}.$$

Also,

$$\mathbf{step}(\boldsymbol{a}_{r-1}(\boldsymbol{x})) \in \mathbb{R}^{1\times d_{r-1}}.$$

These two rows have identical dimensions, so their componentwise product is defined. The result has the same shape as $\boldsymbol{a}_{r-1}(\boldsymbol{x})$, as required for the gradient of J_{r-1} at that preactivation.

Returning to the scalar example. We can display the same distinction between the two products in the scalar backward step. Recall from Lecture 1 that a scalar can be identified with a 1×1 matrix. For two such matrices, ordinary matrix multiplication and componentwise multiplication both multiply their single entries.

Under this identification, transposing a scalar also leaves it unchanged:

$$w_r^{\mathsf{T}} = w_r, \qquad \phi_{r-1}(x)^{\mathsf{T}} = \phi_{r-1}(x).$$

We can therefore include these transposes explicitly to display the same structure as in the vector formulas. For $r = 2, 3$, our earlier backward formula becomes

$$\begin{aligned} J'_{r-1}(a_{r-1}(x)) &= J'_r(a_r(x))\, w_r\, \operatorname{step}(a_{r-1}(x)) \\ &= \Big(J'_r(a_r(x)) \cdot w_r^{\mathsf{T}}\Big) \odot \operatorname{step}(a_{r-1}(x)). \end{aligned} \tag{20}$$

Both multiplications come from the chain rule. The first accounts for the affine operation; the second accounts for ReLU. For scalars, the two products coincide and the transposes leave the values unchanged.

Figures 7 and 8 show the scalar and vector steps in the same layout. In the vector case, the scalar intermediate derivative is replaced by a gradient row, the scalar weight by a weight matrix, and the scalar parameter derivatives by arrays of partial derivatives. Each parameter gradient has the same shape as its corresponding parameter.

Purple $\cdot$ marks ordinary matrix multiplication and green $\odot$ marks componentwise multiplication in both diagrams and in the vector examples that follow.

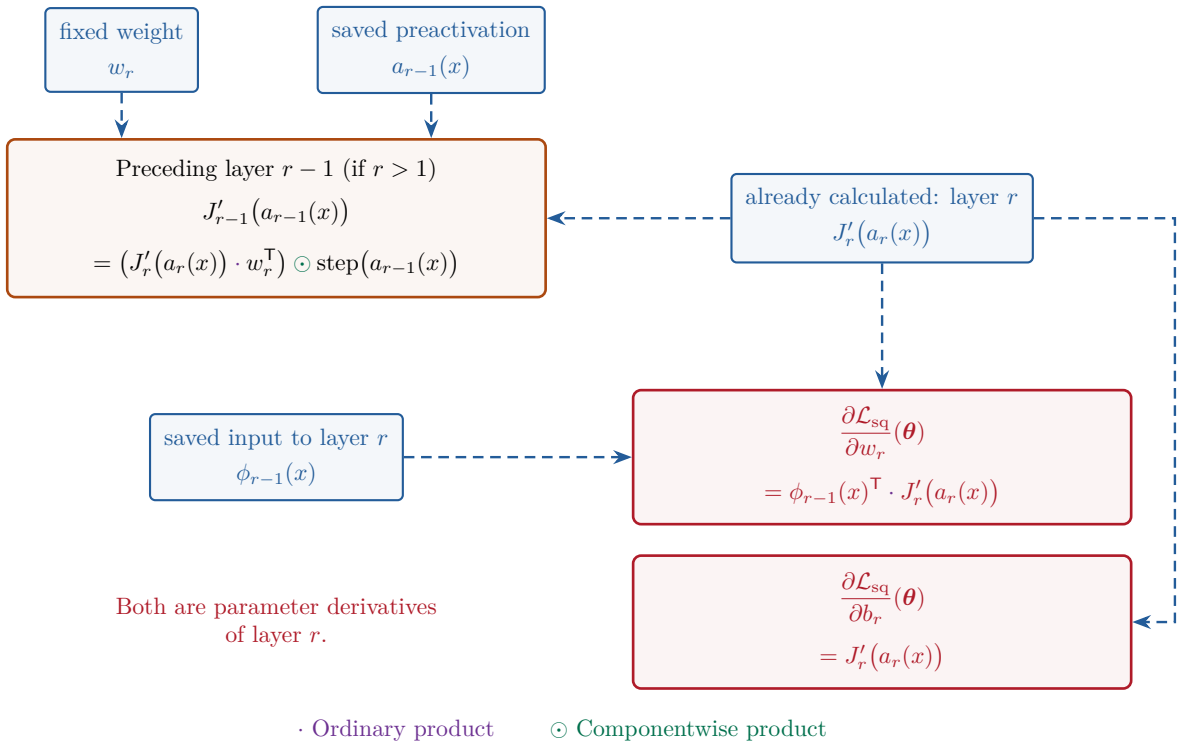


Figure 7: The scalar backward step from Figure 3, with products and transposes displayed to match the vector formulas. Purple $\cdot$ marks ordinary matrix multiplication, and green $\odot$ marks componentwise multiplication. Scalars are identified with 1×1 matrices: both products multiply single entries, and transposition leaves them unchanged. Hidden preactivations are assumed nonzero.

In the vector diagram, the blue gradient at layer r has already been calculated. Together with the saved layer input, it gives the two red parameter gradients below it. Together with the weights and the saved preactivations, it gives the orange gradient for layer $r-1$ to its left. At the next backward step, that orange result becomes the blue input.

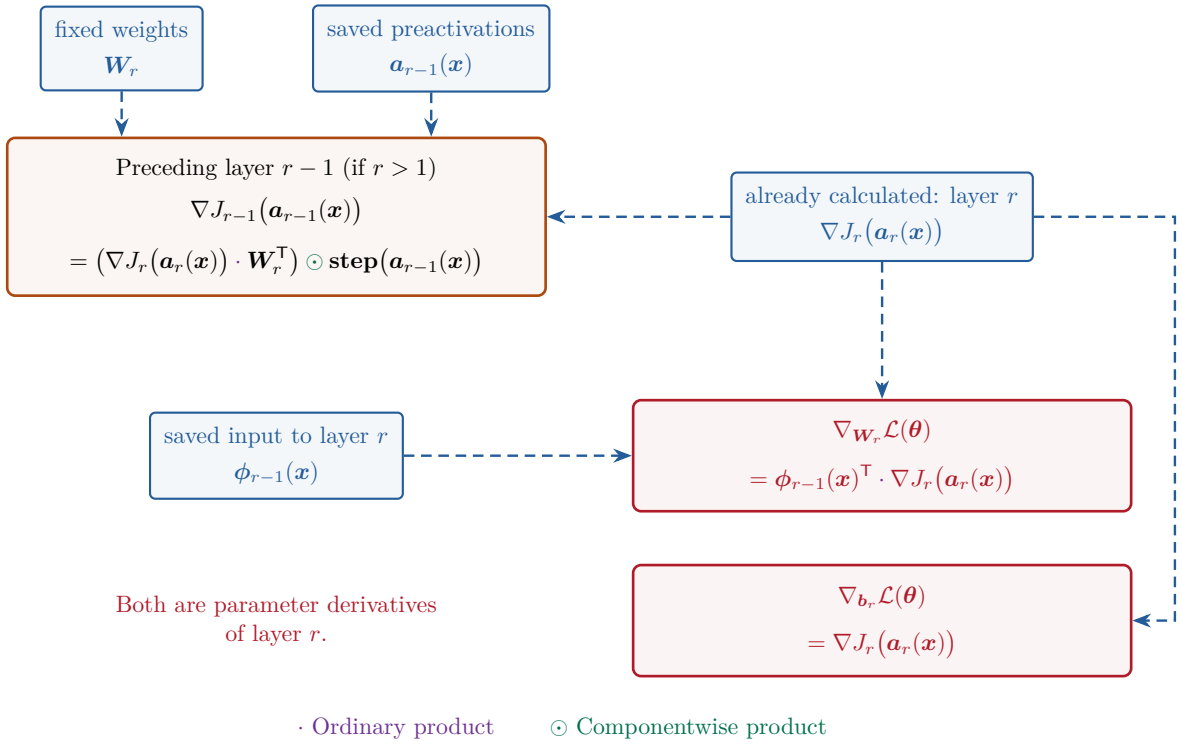


Figure 8: The backward pass proceeds from right to left. The blue gradient at layer r is reused to calculate the two red parameter gradients below it and the orange gradient for layer $r-1$ to its left. The orange result is reused at the next backward step. As in Figure 7, purple $\cdot$ marks ordinary matrix multiplication and green $\odot$ marks componentwise multiplication. Here **step** applies the scalar step function to each coordinate. Every hidden preactivation coordinate is assumed nonzero.

2.5 Completing the example from Lecture 4

We now apply these calculations to the network from Lecture 4. It has two input coordinates, one hidden layer of width three, and one output unit. Its parameters are

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & 2 & -1 \\ -1 & 0 & 1 \end{pmatrix}, \quad \boldsymbol{b}_1 = \begin{pmatrix} 0 & 1 & 0 \end{pmatrix}, \qquad \boldsymbol{W}_2 = \begin{pmatrix} 2 \\ -1 \\ 1/2 \end{pmatrix}, \quad b_2 = -1. \tag{21}$$

The output layer has one unit. We therefore identify its 1×1 bias row with its single entry, which we denote by b_2.

For the input $\boldsymbol{x} = \begin{pmatrix} 1 & 2 \end{pmatrix}$, the forward pass gives

$$\begin{aligned} \boldsymbol{\phi}_0(\boldsymbol{x}) &= \begin{pmatrix} 1 & 2 \end{pmatrix}, \\ \boldsymbol{a}_1(\boldsymbol{x}) &= \boldsymbol{\phi}_0(\boldsymbol{x}) \cdot \boldsymbol{W}_1 + \boldsymbol{b}_1 = \begin{pmatrix} -1 & 3 & 1 \end{pmatrix}, \\ \boldsymbol{\phi}_1(\boldsymbol{x}) &= \mathbf{ReLU}(\boldsymbol{a}_1(\boldsymbol{x})) = \begin{pmatrix} 0 & 3 & 1 \end{pmatrix}, \\ \boldsymbol{a}_2(\boldsymbol{x}) &= \boldsymbol{\phi}_1(\boldsymbol{x}) \cdot \boldsymbol{W}_2 + b_2 = \begin{pmatrix} -7/2 \end{pmatrix}. \end{aligned}$$

The output row has one coordinate, so the prediction is $f_{\boldsymbol{\theta}}(\boldsymbol{x}) = -7/2$. We take $y = -3$, giving $\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = (-7/2 + 3)^2 = 1/4$. Figure 9 shows these calculations in the same arrangement as the scalar forward pass. The boxes along the top contain rows, except for the final scalar loss. We save the layer inputs and preactivations and keep the parameters fixed throughout the backward pass.

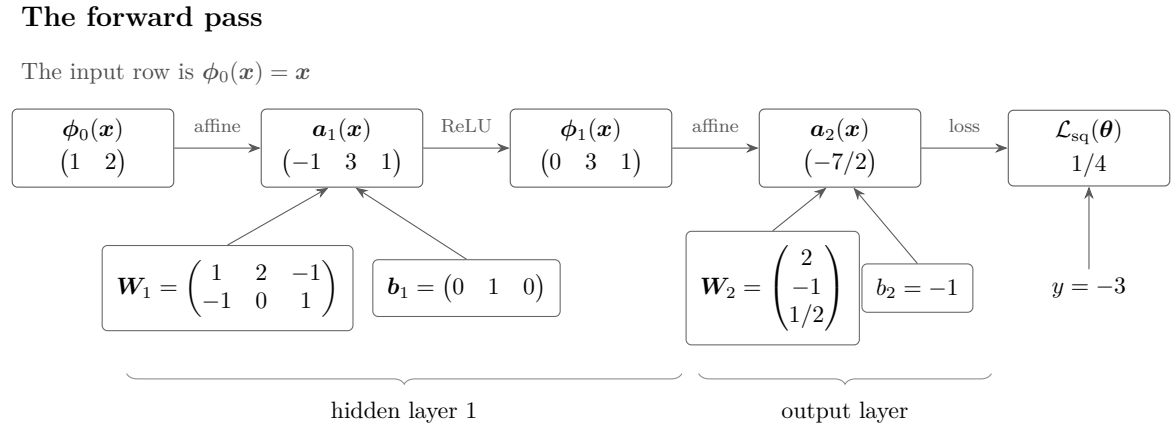


Figure 9: The forward pass for the network in (21). The input has two coordinates, the hidden layer produces three features, and the output row has one coordinate. The weights and biases enter their affine calculations, while the target enters the squared loss. These forward values are kept for the backward pass.

We start at the output layer, which has only one unit. Its preactivation is therefore a row with a single coordinate:

$$\boldsymbol{a}_2(\boldsymbol{x}) = \begin{pmatrix} -7/2 \end{pmatrix} \in \mathbb{R}^{1\times 1}.$$

In particular, $[\boldsymbol{a}_2(\boldsymbol{x})]_1 = -7/2$. We keep the row notation to use the same formulas as for layers with several units.

The remaining function is

$$J_2 : \mathbb{R}^{1\times 1} \longrightarrow \mathbb{R}, \qquad J_2(\boldsymbol{u}) = (u_1 - y)^2, \qquad \boldsymbol{u} = (u_1).$$

Differentiating with respect to its single input coordinate and evaluating at the saved output gives a gradient row with a single entry:

$$\nabla J_2(\boldsymbol{a}_2(\boldsymbol{x})) = \begin{pmatrix} 2 \cdot ([\boldsymbol{a}_2(\boldsymbol{x})]_1 - y) \end{pmatrix} = \begin{pmatrix} 2 \cdot (-7/2 + 3) \end{pmatrix} = \begin{pmatrix} -1 \end{pmatrix}.$$

The single entry of this row is the derivative with respect to b_2. To obtain the weight gradient, we multiply the row on the left by $\boldsymbol{\phi}_1(\boldsymbol{x})^\mathsf{T}$, using the features saved during the forward pass. Figure 10 writes out both parameter derivatives and shows their inputs separately.

1. Output layer

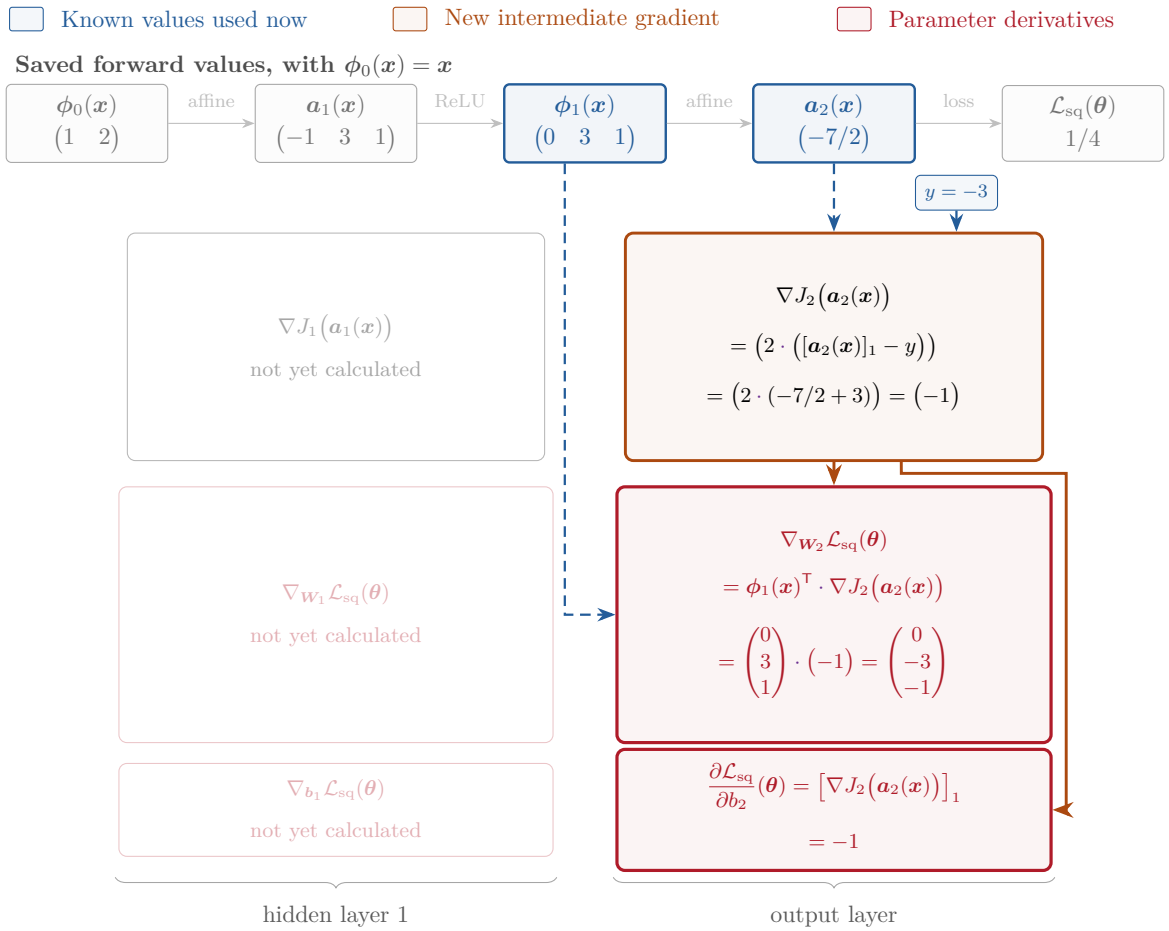


Figure 10: Step 1: calculate the output layer's derivatives. The saved output and target give the orange row. That row and the saved feature row $\boldsymbol{\phi}_1(\boldsymbol{x})$ both enter the weight-gradient calculation. The single entry of the orange row also gives the derivative with respect to the scalar bias b_2.

We then move to the hidden layer. As shown in Figure 11, multiplying the previously calculated $\nabla J_2(\boldsymbol{a}_2(\boldsymbol{x}))$ by $\boldsymbol{W}_2^\mathsf{T}$ gives $\begin{pmatrix}-2 & 1 & -1/2\end{pmatrix}$. The saved preactivation row is $\boldsymbol{a}_1(\boldsymbol{x}) = \begin{pmatrix}-1 & 3 & 1\end{pmatrix}$, so $\mathbf{step}(\boldsymbol{a}_1(\boldsymbol{x})) = \begin{pmatrix}0 & 1 & 1\end{pmatrix}$. The componentwise product therefore gives

$$\nabla J_1(\boldsymbol{a}_1(\boldsymbol{x})) = \begin{pmatrix}-2 & 1 & -1/2\end{pmatrix} \odot \begin{pmatrix}0 & 1 & 1\end{pmatrix} = \begin{pmatrix}0 & 1 & -1/2\end{pmatrix}.$$

The negative last entry is retained because its preactivation is positive.

This row gives the hidden bias gradient. Multiplying it on the left by the saved input $\boldsymbol{\phi}_0(\boldsymbol{x})^\mathsf{T}$ also gives the hidden weight gradient:

$$\begin{aligned}\nabla_{\boldsymbol{W}_1}\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) &= \boldsymbol{\phi}_0(\boldsymbol{x})^\mathsf{T} \cdot \nabla J_1(\boldsymbol{a}_1(\boldsymbol{x})) = \begin{pmatrix}0 & 1 & -1/2\\ 0 & 2 & -1\end{pmatrix},\\ \nabla_{\boldsymbol{b}_1}\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) &= \nabla J_1(\boldsymbol{a}_1(\boldsymbol{x})) = \begin{pmatrix}0 & 1 & -1/2\end{pmatrix}.\end{aligned} \tag{22}$$

The first weight-gradient column and first bias derivative are zero because small changes in those parameters keep the first preactivation negative and its activation zero.

2. Hidden layer 1

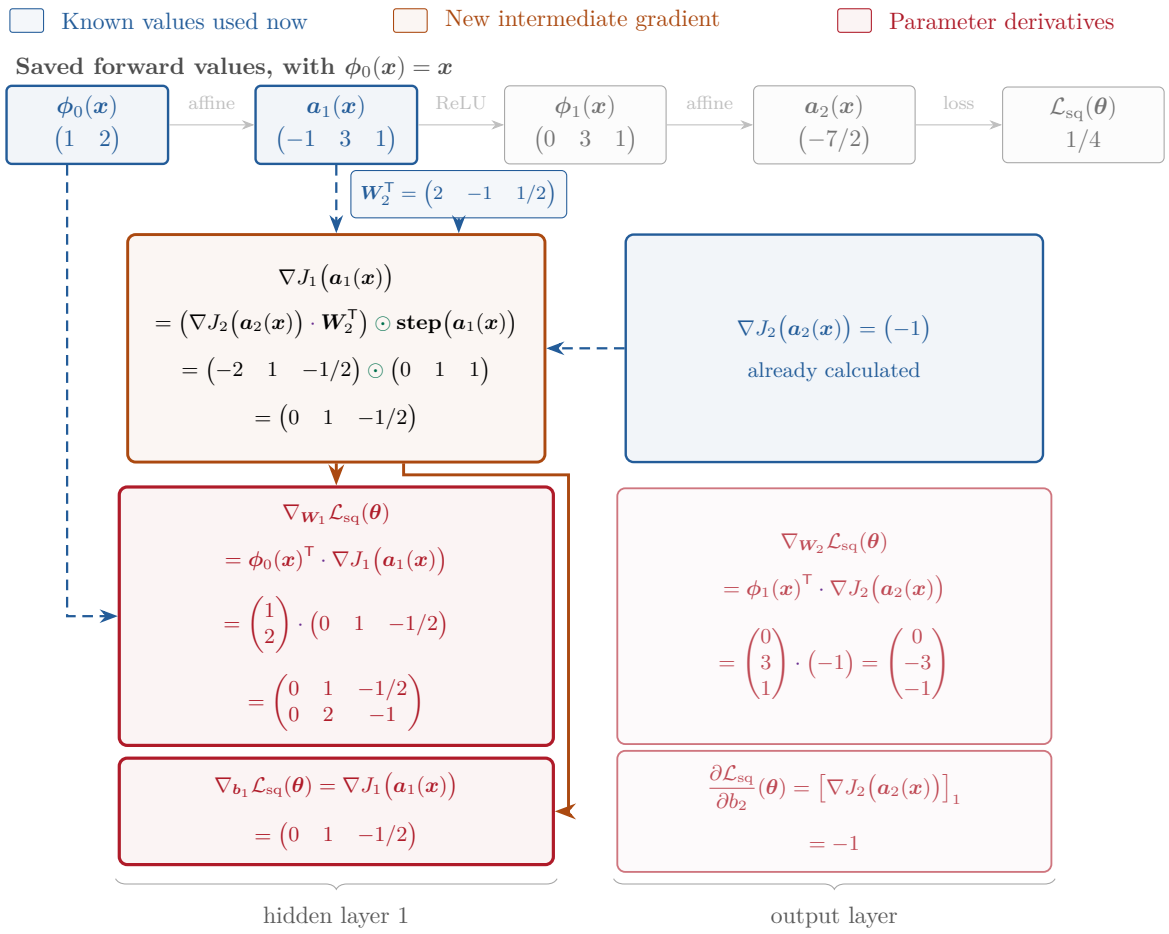


Figure 11: Step 2: calculate the hidden layer's derivatives. The blue row from the preceding step is multiplied by $\boldsymbol{W}_2^\mathsf{T}$. The componentwise product with $\mathbf{step}(\boldsymbol{a}_1(\boldsymbol{x})) = \begin{pmatrix}0 & 1 & 1\end{pmatrix}$ gives the orange row. This row gives the bias gradient and, together with $\boldsymbol{\phi}_0(\boldsymbol{x})$, the weight gradient. Every parameter derivative is now shown in red.

3 Starting from the appropriate loss

The task determines the function J_L that turns the output row into a loss, and therefore the initial backward row $\nabla J_L(\boldsymbol{a}_L(\boldsymbol{x}))$. After this initialisation, the parameter-gradient formulas and the earlier-layer recursion are the same for all three tasks. The following formulas are for one fixed observation. Their derivations are given in Appendix A.

3.1 Regression

For a scalar target $y \in \mathbb{R}$, we use one output, so $d_L = 1$. The remaining function is

$$J_L(\boldsymbol{u}) = \ell_{\text{sq}}(u_1, y) = (u_1 - y)^2.$$

The initial backward row has the single entry

$$[\nabla J_L(\boldsymbol{a}_L(\boldsymbol{x}))]_1 = 2 \cdot \big(f_{\boldsymbol{\theta}}(\boldsymbol{x}) - y\big). \tag{23}$$

This is the value used in both numerical examples.

3.2 Binary classification

For $y \in \{0, 1\}$, we again use one output, but it is now a score, or *logit*. The predicted probability is $p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sigma(f_{\boldsymbol{\theta}}(\boldsymbol{x}))$. The remaining function includes both the sigmoid and the logistic loss:

$$J_L(\boldsymbol{u}) = \ell_{\log}(\sigma(u_1), y).$$

The initial backward row has the single entry

$$[\nabla J_L(\boldsymbol{a}_L(\boldsymbol{x}))]_1 = p_{\boldsymbol{\theta}}(\boldsymbol{x}) - y. \tag{24}$$

This formula already includes the sigmoid derivative. We do not multiply by it again in the backward loop.

3.3 Multiclass classification

For K classes, $y \in \{1, \ldots, K\}$, and the output width is $d_L = K$. Softmax turns the score row into the probability row $\boldsymbol{p}_{\boldsymbol{\theta}}(\boldsymbol{x}) = \text{softmax}(\boldsymbol{a}_L(\boldsymbol{x}))$. The remaining function is

$$J_L(\boldsymbol{u}) = \ell_{\text{multi}}(\text{softmax}(\boldsymbol{u}), y).$$

The initial backward row has entries

$$[\nabla J_L(\boldsymbol{a}_L(\boldsymbol{x}))]_k = p_{\boldsymbol{\theta},k}(\boldsymbol{x}) - \delta_{ky}, \qquad k = 1, \ldots, K, \tag{25}$$

where δ_{ky} is the Kronecker delta introduced in Lecture 1. Thus we take the probability row and subtract one from the entry corresponding to the observed class y, leaving all other entries unchanged.

4 From one observation to the whole dataset

So far, we have calculated the loss derivatives for one fixed observation. Now consider a fixed dataset $(\boldsymbol{x}_i, y_i)$, $i = 1, \ldots, N$, with the same network parameters $\boldsymbol{\theta}$ used for every observation.

For each observation, the remaining functions J_r use its target y_i; this dependence remains implicit.

For each i, let $\mathcal{L}_i$ be the loss function for the fixed observation $(\boldsymbol{x}_i, y_i)$, using the appropriate choice from Section 3. Its argument is the parameter vector $\boldsymbol{\theta}$. For example, in regression,

$$\mathcal{L}_i(\boldsymbol{\theta}) = \ell_{\mathrm{sq}}(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i), y_i) = \big(f_{\boldsymbol{\theta}}(\boldsymbol{x}_i) - y_i\big)^2.$$

As in Lecture 4, the empirical loss for this dataset is the function

$$\mathcal{L}(\boldsymbol{\theta}) \coloneqq \sum_{i=1}^{N} \mathcal{L}_i(\boldsymbol{\theta}). \tag{26}$$

Under the differentiability assumptions used above, differentiating this sum gives

$$\nabla\mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^{N} \nabla\mathcal{L}_i(\boldsymbol{\theta}). \tag{27}$$

Each term on the right is calculated by the forward and backward passes described in this lecture. To obtain the gradient for the whole dataset:

1. Start with an accumulated gradient whose entries are zero.
2. For each observation, calculate its loss gradient and add it to the accumulated gradient.
3. After processing all observations, use the accumulated gradient to make one gradient-descent update.

Keep the parameters fixed throughout the first two steps: every term in (27) must be evaluated at the same $\boldsymbol{\theta}$. Update the parameters only after the sum is complete.

5 Why this calculation is efficient

Consider one observation and a network of the form studied in this lecture, with P scalar weights and biases. A forward pass followed by backpropagation calculates the loss and all P parameter derivatives with

Time: $O(P)$	Memory: $O(P)$.

Here $O(P)$ means at most proportional to the number of parameters, up to a constant factor. The memory includes the parameters, the gradient and the saved forward values. These one-observation costs are optimal up to constant factors when we explicitly calculate and store the full gradient: writing its P entries already takes at least P operations, and storing them requires P numbers.

For the whole dataset. Repeating this calculation for N observations and adding their gradients takes $O(NP)$ time. We can process the observations one at a time: after adding an observation's gradient to the accumulated gradient, we can discard its saved forward values. The working memory then remains $O(P)$, excluding storage of the dataset itself.

An alternative: one parameter derivative at a time. For one fixed observation, we could calculate the parameter derivatives separately, repeating the chain rule calculation for each parameter.

For example, consider a scalar network with L layers, each containing one weight and one bias. There are therefore $P = 2L$ parameters. First calculate and save the forward values.

To differentiate the loss with respect to a parameter in layer r, start at that layer and propagate the derivative towards the loss. Earlier quantities do not depend on the chosen parameter, so their derivatives with respect to it are zero.

For either parameter in the first layer, this calculation passes through all L layers. For either parameter in the second layer, it passes through $L - 1$ layers, and so on. If we repeat the calculation separately for every weight and bias, the total number of layer visits is

$$2\big(L + (L-1) + \cdots + 1\big) = L(L+1).$$

Each visit requires differentiating one scalar affine operation and, for a hidden layer, one ReLU. At the output, we also differentiate the loss. Each visit therefore costs a bounded number of arithmetic operations. Since $P = 2L$, the total cost is $O(P^2)$.

The repeated work occurs in the later layers: we differentiate the same remaining calculations again for each earlier parameter. Backpropagation calculates their derivatives once and reuses them. It obtains all P parameter derivatives with one forward pass and one backward pass, taking $O(P)$ time in total.

For a dataset with N observations, repeating these procedures for every observation gives $O(NP^2)$ time for the separate calculations, compared with $O(NP)$ for backpropagation.

6 Exercises

The following exercises practise a complete forward and backward pass for regression, binary classification and classification with three classes. Use the formulas from the main text; the optional appendix is not required. Keep every parameter at its forward-pass value until all the gradients have been calculated.

1. **A complete numerical backward pass.** Use the Lecture 4 network

$$\begin{aligned} \boldsymbol{a}_1(\boldsymbol{x}) &= \boldsymbol{x} \cdot \boldsymbol{W}_1 + \boldsymbol{b}_1, \qquad & \boldsymbol{\phi}_1(\boldsymbol{x}) &= \mathbf{ReLU}\big(\boldsymbol{a}_1(\boldsymbol{x})\big), \\ f_{\boldsymbol{\theta}}(\boldsymbol{x}) &= \boldsymbol{\phi}_1(\boldsymbol{x}) \cdot \boldsymbol{W}_2 + b_2. \end{aligned}$$

   with

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & 2 & -1 \\ -1 & 0 & 1 \end{pmatrix}, \quad \boldsymbol{b}_1 = \begin{pmatrix} 0 & 1 & 0 \end{pmatrix}, \qquad \boldsymbol{W}_2 = \begin{pmatrix} 2 \\ -1 \\ 1/2 \end{pmatrix}, \quad b_2 = -1.$$

   Take $\boldsymbol{x} = (2, 0)$ and the regression target $y = 1$. For this fixed observation, the objective is $\mathcal{L}_{\text{sq}}(\boldsymbol{\theta}) = (f_{\boldsymbol{\theta}}(\boldsymbol{x}) - y)^2$. The final preactivation row is $\boldsymbol{a}_2(\boldsymbol{x}) = (f_{\boldsymbol{\theta}}(\boldsymbol{x}))$.

   (a) Compute $\boldsymbol{a}_1(\boldsymbol{x})$, $\boldsymbol{\phi}_1(\boldsymbol{x})$, $f_{\boldsymbol{\theta}}(\boldsymbol{x})$, and the squared loss. State the shapes of both weight matrices and the hidden rows.

   (b) Write the remaining-loss functions $J_1 : \mathbb{R}^{1\times 3} \to \mathbb{R}$ and $J_2 : \mathbb{R}^{1\times 1} \to \mathbb{R}$ for this network. Calculate $\nabla J_2(\boldsymbol{a}_2(\boldsymbol{x}))$ and then $\nabla J_1(\boldsymbol{a}_1(\boldsymbol{x}))$. Use these rows to find the gradients of $\mathcal{L}_{\text{sq}}$ with respect to every weight and bias, evaluated at $\boldsymbol{\theta}$. Keep every parameter at its forward-pass value throughout this calculation.

(c) Explain how the negative third preactivation affects the backward pass. Explain separately why the second row of the $\boldsymbol{W}_1$ gradient is zero for this observation.

(d) Only after completing the backward pass, write one gradient descent update of all parameters using a learning rate $\eta > 0$.

2. **Binary classification.** Consider a network with one input coordinate, two ReLU hidden units and one output logit:

$$\begin{aligned} \boldsymbol{a}_1(\boldsymbol{x}) &= \boldsymbol{x} \cdot \boldsymbol{W}_1 + \boldsymbol{b}_1, \qquad & \boldsymbol{\phi}_1(\boldsymbol{x}) &= \mathbf{ReLU}(\boldsymbol{a}_1(\boldsymbol{x})), \\ f_{\boldsymbol{\theta}}(\boldsymbol{x}) &= \boldsymbol{\phi}_1(\boldsymbol{x}) \cdot \boldsymbol{W}_2 + b_2. \end{aligned}$$

Use

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & 2 \end{pmatrix}, \quad \boldsymbol{b}_1 = \begin{pmatrix} 0 & 0 \end{pmatrix}, \qquad \boldsymbol{W}_2 = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad b_2 = 0,$$

with input row $\boldsymbol{x} = (1) \in \mathbb{R}^{1\times 1}$ and target $y = 1$. The predicted class-1 probability and the loss for this fixed observation are

$$p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sigma(f_{\boldsymbol{\theta}}(\boldsymbol{x})), \qquad \mathcal{L}_{\log}(\boldsymbol{\theta}) = \ell_{\log}(p_{\boldsymbol{\theta}}(\boldsymbol{x}), y).$$

As in Exercise 1, write the final preactivation as the row $\boldsymbol{a}_2(\boldsymbol{x}) = (f_{\boldsymbol{\theta}}(\boldsymbol{x}))$.

(a) Compute the hidden preactivations and activations, the output logit, the class-1 probability and the logistic loss. Keep the loss in exact form.

(b) Write $J_2 : \mathbb{R}^{1\times 1} \to \mathbb{R}$, including the sigmoid in this remaining function. Use (24) to calculate $\nabla J_2(\boldsymbol{a}_2(\boldsymbol{x}))$, then find $\nabla_{\boldsymbol{W}_2}\mathcal{L}_{\log}(\boldsymbol{\theta})$ and $\frac{\partial \mathcal{L}_{\log}}{\partial b_2}(\boldsymbol{\theta})$.

(c) Calculate $\nabla J_1(\boldsymbol{a}_1(\boldsymbol{x}))$ using the saved preactivations and $\boldsymbol{W}_2$. Use this row to find $\nabla_{\boldsymbol{W}_1}\mathcal{L}_{\log}(\boldsymbol{\theta})$ and $\nabla_{\boldsymbol{b}_1}\mathcal{L}_{\log}(\boldsymbol{\theta})$.

3. **Classification with three classes.** Keep the input and the first-layer parameters from Exercise 2. Replace the output layer by three logits, using

$$\boldsymbol{W}_2 = \begin{pmatrix} 1 & -1 & 0 \\ 0 & 1 & -1 \end{pmatrix}, \qquad \boldsymbol{b}_2 = \begin{pmatrix} -1 & -1 & 2 \end{pmatrix}.$$

The network output and probability row are now

$$\begin{aligned} \boldsymbol{a}_2(\boldsymbol{x}) = \boldsymbol{f}_{\boldsymbol{\theta}}(\boldsymbol{x}) &= \boldsymbol{\phi}_1(\boldsymbol{x}) \cdot \boldsymbol{W}_2 + \boldsymbol{b}_2, \\ \boldsymbol{p}_{\boldsymbol{\theta}}(\boldsymbol{x}) &= \operatorname{softmax}(\boldsymbol{a}_2(\boldsymbol{x})). \end{aligned}$$

The class labels are $1, 2, 3$, and the observed class is $y = 2$. For this fixed observation, use the cross-entropy objective

$$\mathcal{L}_{\text{multi}}(\boldsymbol{\theta}) = \ell_{\text{multi}}(\boldsymbol{p}_{\boldsymbol{\theta}}(\boldsymbol{x}), y) = -\log p_{\boldsymbol{\theta},2}(\boldsymbol{x}).$$

You may reuse the hidden preactivations and activations from Exercise 2.

(a) Compute the three logits, their softmax probabilities and the cross-entropy loss. Keep the loss in exact form.

(b) Write $J_2 : \mathbb{R}^{1\times 3} \to \mathbb{R}$, including softmax in this remaining function. Use (25) to calculate $\nabla J_2(\boldsymbol{a}_2(\boldsymbol{x}))$, then find $\nabla_{\boldsymbol{W}_2}\mathcal{L}_{\text{multi}}(\boldsymbol{\theta})$ and $\nabla_{\boldsymbol{b}_2}\mathcal{L}_{\text{multi}}(\boldsymbol{\theta})$.

(c) Compute $\nabla J_2(\boldsymbol{a}_2(\boldsymbol{x})) \cdot \boldsymbol{W}_2^{\mathsf{T}}$. Write out the three contributions to its first entry before adding them. Apply the componentwise ReLU factors to obtain $\nabla J_1(\boldsymbol{a}_1(\boldsymbol{x}))$, then calculate $\nabla_{\boldsymbol{W}_1}\mathcal{L}_{\text{multi}}(\boldsymbol{\theta})$ and $\nabla_{\boldsymbol{b}_1}\mathcal{L}_{\text{multi}}(\boldsymbol{\theta})$.

A Optional: derivation of the backpropagation formulas

This appendix justifies the formulas used in the main text. We state the multivariable chain rule, then use it to derive the backward recursion, the parameter gradients and the initial backward row for each loss.

A.1 The multivariable chain rule

Let $F : \mathbb{R}^{1\times n} \to \mathbb{R}$ have independent input coordinates $z_1, \dots, z_n$, and let $\boldsymbol{u} : \mathbb{R} \to \mathbb{R}^{1\times n}$ have coordinate functions $u_1, \dots, u_n$. Define the scalar composition

$$g = F \circ \boldsymbol{u}, \qquad g(t) = F(\boldsymbol{u}(t)), \qquad \boldsymbol{u}(t) = \begin{pmatrix} u_1(t) & \cdots & u_n(t) \end{pmatrix}.$$

Write $\partial_j F$ for the partial derivative of F with respect to z_j, holding its other coordinates fixed. If $\boldsymbol{u}$ is differentiable at t and F is differentiable at $\boldsymbol{u}(t)$, the chain rule gives

$$g'(t) = \sum_{j=1}^{n} (\partial_j F)(\boldsymbol{u}(t)) \cdot u_j'(t). \tag{28}$$

Changing t may change every input coordinate of F. The sum adds their contributions, evaluating every partial derivative at the same row $\boldsymbol{u}(t)$.

Both the gradient $\nabla F(\boldsymbol{u}(t))$ and the derivative row

$$\boldsymbol{u}'(t) = \begin{pmatrix} u_1'(t) & \cdots & u_n'(t) \end{pmatrix}$$

have shape $1 \times n$. Consequently, the same formula is

$$g'(t) = \nabla F(\boldsymbol{u}(t)) \cdot (\boldsymbol{u}'(t))^{\mathsf{T}}. \tag{29}$$

The product has shape 1×1, which we identify with its scalar entry. To calculate a partial derivative of a composition with several inputs, apply this rule while varying one input coordinate and holding the others fixed.

A.2 The backward recursion

We now justify the formulas in Section 2. The input, target and current parameters are fixed, and all hidden preactivation coordinates are assumed nonzero. We use the remaining functions defined in (14) and the chain rule from Section A.1.

Suppose we have already calculated $\nabla J_{r+1}(\boldsymbol{a}_{r+1}(\boldsymbol{x}))$. To find $\nabla J_r(\boldsymbol{a}_r(\boldsymbol{x}))$, differentiate (14). Write $\boldsymbol{v}$ for the row argument of J_{r+1}. Its k-th argument in the composition is

$$\left[\mathbf{ReLU}(\boldsymbol{u}) \cdot \boldsymbol{W}_{r+1} + \boldsymbol{b}_{r+1}\right]_k = \sum_{j=1}^{d_r} \mathrm{ReLU}(u_j) \cdot (\boldsymbol{W}_{r+1})_{jk} + (\boldsymbol{b}_{r+1})_k.$$

For $\boldsymbol{u}$ sufficiently close to $\boldsymbol{a}_r(\boldsymbol{x})$, the nonzero-preactivation assumption ensures that all functions in this composition are differentiable. Changing u_j changes the k-th argument at rate $\mathrm{ReLU}'(u_j) \cdot (\boldsymbol{W}_{r+1})_{jk}$. Since it may change every argument of J_{r+1}, the multivariable chain rule gives

$$\left(\frac{\partial J_r}{\partial u_j}\right)(\boldsymbol{u}) = \sum_{k=1}^{d_{r+1}} \left(\frac{\partial J_{r+1}}{\partial v_k}\right)(\mathbf{ReLU}(\boldsymbol{u}) \cdot \boldsymbol{W}_{r+1} + \boldsymbol{b}_{r+1}) \cdot \mathrm{ReLU}'(u_j) \cdot (\boldsymbol{W}_{r+1})_{jk}. \tag{30}$$

Now substitute $\boldsymbol{u} = \boldsymbol{a}_r(\boldsymbol{x})$. The row passed to J_{r+1} becomes $\boldsymbol{a}_{r+1}(\boldsymbol{x})$, so its evaluated partial derivatives are exactly the entries of $\nabla J_{r+1}(\boldsymbol{a}_{r+1}(\boldsymbol{x}))$. Using $\mathrm{ReLU}'([\boldsymbol{a}_r(\boldsymbol{x})]_j) = \mathrm{step}([\boldsymbol{a}_r(\boldsymbol{x})]_j)$, the individual factors are therefore

$$[\nabla J_r(\boldsymbol{a}_r(\boldsymbol{x}))]_j = \sum_{k=1}^{d_{r+1}} \underbrace{[\nabla J_{r+1}(\boldsymbol{a}_{r+1}(\boldsymbol{x}))]_k}_{\text{already calculated}} \cdot \underbrace{(\boldsymbol{W}_{r+1})_{jk}}_{\text{affine derivative}} \cdot \underbrace{\mathrm{step}([\boldsymbol{a}_r(\boldsymbol{x})]_j)}_{\text{ReLU derivative}}.$$

As in the scalar calculation, the first factor is already available. The other two factors come from the affine operation and ReLU encountered as we work backwards. We now add one contribution for each next-layer coordinate k. The ReLU factor is the same in every term, so we can take it outside the sum:

$$[\nabla J_r(\boldsymbol{a}_r(\boldsymbol{x}))]_j = \mathrm{step}([\boldsymbol{a}_r(\boldsymbol{x})]_j) \cdot \sum_{k=1}^{d_{r+1}} (\boldsymbol{W}_{r+1})_{jk} \cdot [\nabla J_{r+1}(\boldsymbol{a}_{r+1}(\boldsymbol{x}))]_k. \tag{31}$$

Equation (31) calculates one coordinate. To calculate all of them, notice that its sums are the entries of a single matrix product:

$$\left[\nabla J_{r+1}(\boldsymbol{a}_{r+1}(\boldsymbol{x})) \cdot \boldsymbol{W}_{r+1}^{\mathsf{T}}\right]_j = \sum_{k=1}^{d_{r+1}} [\nabla J_{r+1}(\boldsymbol{a}_{r+1}(\boldsymbol{x}))]_k \cdot (\boldsymbol{W}_{r+1})_{jk}. \tag{32}$$

For each j, the product pairs the derivatives in $\nabla J_{r+1}(\boldsymbol{a}_{r+1}(\boldsymbol{x}))$ with the weights connecting unit j to the next layer. This is the same multiplication and addition that appeared in (29), now carried out for every coordinate of the preceding row. The evaluated gradient has shape $1 \times d_{r+1}$, while $\boldsymbol{W}_{r+1}^{\mathsf{T}}$ has shape $d_{r+1} \times d_r$. Their product is therefore a row of shape $1 \times d_r$.

Multiplying this row componentwise by $\mathbf{step}(\boldsymbol{a}_r(\boldsymbol{x}))$ gives all coordinates of $\nabla J_r(\boldsymbol{a}_r(\boldsymbol{x}))$. Relabelling the current layer $r+1$ as r gives (19).

A.3 The parameter gradients

Fix a layer r and consider the weight $(\boldsymbol{W}_r)_{kj}$. The input feature row $\boldsymbol{\phi}_{r-1}(\boldsymbol{x})$ is calculated by the preceding layers, so it does not depend on this weight. The function J_r does not depend on it either, because it uses only the parameters of subsequent layers and the target.

This weight multiplies feature k when we calculate preactivation j. It does not appear in the calculation of any other preactivation in the same layer. For $\ell = 1, \ldots, d_r$, we therefore have

$$\frac{\partial [\boldsymbol{a}_r(\boldsymbol{x})]_\ell}{\partial (\boldsymbol{W}_r)_{kj}} = \begin{cases} [\boldsymbol{\phi}_{r-1}(\boldsymbol{x})]_k, & \ell = j, \\ 0, & \ell \neq j. \end{cases}$$

These derivatives are evaluated at the current parameters, with the input, target and all other parameters held fixed.

We have written the loss as $\mathcal{L}(\boldsymbol{\theta}) = J_r(\boldsymbol{a}_r(\boldsymbol{x}))$. Applying the multivariable chain rule to this composition gives

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial (\boldsymbol{W}_r)_{kj}}(\boldsymbol{\theta}) &= \sum_{\ell=1}^{d_r} \left(\frac{\partial J_r}{\partial u_\ell}\right)(\boldsymbol{a}_r(\boldsymbol{x})) \cdot \frac{\partial [\boldsymbol{a}_r(\boldsymbol{x})]_\ell}{\partial (\boldsymbol{W}_r)_{kj}} \\ &= \left(\frac{\partial J_r}{\partial u_j}\right)(\boldsymbol{a}_r(\boldsymbol{x})) \cdot [\boldsymbol{\phi}_{r-1}(\boldsymbol{x})]_k. \end{aligned} \tag{33}$$

Only the term with $\ell = j$ remains, because all the other preactivation derivatives are zero.

For the bias $(\boldsymbol{b}_r)_j$, only preactivation j depends on it, and its derivative is one. The same chain rule therefore gives

$$\frac{\partial \mathcal{L}}{\partial (\boldsymbol{b}_r)_j}(\boldsymbol{\theta}) = \left(\frac{\partial J_r}{\partial u_j}\right)(\boldsymbol{a}_r(\boldsymbol{x})).$$

Thus the same evaluated partial derivative of J_r is reused for the bias and for every weight entering unit j. Each weight derivative also includes the corresponding input feature value.

Collecting the weight derivatives into a matrix and the bias derivatives into a row gives (17). The weight-gradient entry (k, j) is the product of input coordinate k and backward coordinate j, which is exactly the entry of the stated outer product.

A.4 The initial backward row

We now derive the three initialisations stated in Section 3. We differentiate the remaining function J_L with respect to its row argument, then evaluate at the output calculated in the forward pass.

A.4.1 Regression

A scalar target requires $d_L = 1$. We use the network output directly as the prediction, so

$$J_L(\boldsymbol{u}) = \ell_{\text{sq}}(u_1, y) = (u_1 - y)^2, \qquad \left(\frac{\partial J_L}{\partial u_1}\right)(\boldsymbol{u}) = 2 \cdot (u_1 - y).$$

Evaluating at the output gives

$$[\nabla J_L(\boldsymbol{a}_L(\boldsymbol{x}))]_1 = 2 \cdot (f_{\boldsymbol{\theta}}(\boldsymbol{x}) - y). \tag{34}$$

This is the initial value used in both numerical examples.

A.4.2 Binary classification

Here $d_L = 1$ again, but the output is a score, also called a *logit*. We apply the sigmoid to obtain the probability $p_{\boldsymbol{\theta}}(\boldsymbol{x}) = \sigma(f_{\boldsymbol{\theta}}(\boldsymbol{x}))$. The final function must therefore include both the sigmoid and the logistic loss:

$$J_L(\boldsymbol{u}) = \ell_{\log}(\sigma(u_1), y) = -y \cdot \log \sigma(u_1) - (1 - y) \cdot \log(1 - \sigma(u_1)).$$

For an independent probability argument p, the loss has partial derivative

$$\frac{\partial \ell_{\log}}{\partial p}(p, y) = -\frac{y}{p} + \frac{1 - y}{1 - p} = \frac{p - y}{p \cdot (1 - p)}.$$

Since $\sigma'(u_1) = \sigma(u_1) \cdot (1 - \sigma(u_1))$, applying the chain rule gives

$$\begin{aligned}\left(\frac{\partial J_L}{\partial u_1}\right)(\boldsymbol{u}) &= \frac{\partial \ell_{\log}}{\partial p}(\sigma(u_1), y) \cdot \sigma'(u_1) \\ &= \frac{\sigma(u_1) - y}{\sigma(u_1) \cdot (1 - \sigma(u_1))} \cdot \sigma(u_1) \cdot (1 - \sigma(u_1)) = \sigma(u_1) - y.\end{aligned}$$

Thus the initial backward value is

$$[\nabla J_L(\boldsymbol{a}_L(\boldsymbol{x}))]_1 = p_{\boldsymbol{\theta}}(\boldsymbol{x}) - y. \tag{35}$$

This already includes the sigmoid derivative. We use it directly in the output layer's parameter-gradient formulas.

A.4.3 Multiclass classification

For K classes, $d_L = K$. The output contains one score, or logit, per class, and softmax produces the probability row $\boldsymbol{p}_{\boldsymbol{\theta}}(\boldsymbol{x})$. Consequently,

$$J_L(\boldsymbol{u}) = \ell_{\text{multi}}(\text{softmax}(\boldsymbol{u}), y) = -\log \frac{e^{u_y}}{\sum_{j=1}^{K} e^{u_j}} = -u_y + \log \left(\sum_{j=1}^{K} e^{u_j} \right).$$

The derivative of $-u_y$ with respect to u_k is $-\delta_{ky}$, using the Kronecker delta introduced in Lecture 1. Differentiating the logarithm gives

$$\frac{\partial}{\partial u_k} \log \left(\sum_{j=1}^{K} e^{u_j} \right) = \frac{e^{u_k}}{\sum_{j=1}^{K} e^{u_j}} = [\text{softmax}(\boldsymbol{u})]_k.$$

Therefore,

$$\left(\frac{\partial J_L}{\partial u_k} \right) (\boldsymbol{u}) = \frac{e^{u_k}}{\sum_{j=1}^{K} e^{u_j}} - \delta_{ky}.$$

Evaluating at the network's output gives

$$[\nabla J_L(\boldsymbol{a}_L(\boldsymbol{x}))]_k = p_{\boldsymbol{\theta},k}(\boldsymbol{x}) - \delta_{ky}, \qquad k = 1, \ldots, K. \tag{36}$$

This is the gradient with respect to the logits: differentiation through softmax is already included.